VC Generation

Arie Gurfinkel

Contents

1 Overview

This document describes the foundations of verification-condition generation for ctac.

The purpose is not to mirror the current implementation module-by-module. Instead, the goal is to define the mathematical objects, semantic conventions, and proof obligations that a VC generator for TAC should satisfy. The implementation can then be evaluated against this foundation, and future encoder variants can share the same vocabulary.

The small language used in this document is **Tiny TAC**, abbreviated `ttac`. It is a TAC-like core language for explaining VC generation, not a complete description of the implementation input format.

1.1 Scope

We start with loop-free `ttac` programs that have a distinguished entry block and a single assertion site. Multi-assert programs are treated as a preprocessing problem: they may be transformed into an equivalent single-assert program before VC generation.

At this level, a VC generator consumes:

- a control-flow graph of basic blocks,

- commands inside each block,
- a symbol table and sort information,
- `ttac` expressions over scalars and maps,
- the assertion whose failure reachability is being queried.

It produces an SMT formula whose satisfiability answers the assertion-failure question:

$$\mathrm{sat}(\mathrm{VC}(P, A)) \Leftrightarrow \text{there exists an execution of P reaching a failure of A}$$

Thus, an unsatisfiable VC means the assertion is proved for the encoded program semantics.

1.2 Main Questions

The foundational presentation should answer these questions:

- What is the `ttac` program model used by VC generation?
- What does it mean for a block or command to be reachable?
- How are assignments, havoc, assume, branch, and assert commands interpreted?
- How are path conditions represented?
- How are scalar expressions lowered to SMT?
- How are bytemaps and other map-like values modeled?
- Which side conditions are required before VC generation?
- Which transformations are semantic preprocessing, and which are encoding choices?

1.3 Document Plan

The intended structure is:

1. Program model: blocks, commands, symbols, expressions, and control flow.
2. Operational intuition: executions, feasibility, and assertion failure.
3. VC shape: reachability variables, command constraints, and failure query.
4. Expression semantics: arithmetic, bit-vector-like integer domains, and uninterpreted operations.
5. Map semantics: reads, writes, and finite update chains.
6. Preconditions and preprocessing: acyclicity, single assertion, critical edges, and well-formed definitions.
7. Soundness statement: what it means for the generated VC to be sound with respect to `ttac` executions.

1.4 Conventions

We use P for a `ttac` program, B for the set of basic blocks, and entry for the entry block. A block $b \in B$ has a sequence of commands and zero or more successor blocks. Reachability of a block is represented abstractly by a Boolean predicate R_b.

When we write $\mathrm{VC}(P, A)$, A denotes the selected assertion site. The VC is oriented toward bug finding: satisfiable means an assertion-failure execution exists, and unsatisfiable means no such execution exists under the modeled semantics.

2 Syntax and Semantics of the Target Language

We use **Tiny TAC**, abbreviated `ttac`, as the source language for VC generation. Tiny TAC keeps the semantic core explicit and leaves implementation-specific details out of scope.

2.1 Types

Registers have one of three types:

$$\tau ::= \text{bool} \mid \text{int} \mid \text{bytemap}$$

An int is modeled as an SMT Int. A bytemap is an integer-indexed integer map:

$$\text{bytemap} = \text{int} \to \text{int}$$

We write $x : \tau$ to say that register x has type τ.

2.2 Expressions

```
i ::= n
    | x
    | M[i]
    | i + i
    | i - i
    | i * i
    | i / i
    | ite(b, i, i)

b ::= true
    | false
    | c
    | i <= i
    | i < i
    | i == i
    | b == b
    | not b
    | b and b
    | b or b
    | ite(c, b, b)
```

Here x ranges over integer registers and c ranges over bool registers. The operator `/` denotes SMT integer division. The conditional operator is expression-level if-then-else.

```
m ::= M
    | M[i := v]

v ::= i
```

A load is an integer expression; a store produces a new bytemap value:

```
M := havoc
i := havoc
v := havoc
x := M[i]
M2 := M[i := v]
```

2.3 Commands

```
cmd ::= x := e
      | x := havoc
      | x := phi [B1: x1, B2: x2, ...]
```

```
      | assume b
      | assert c
```

The expression e has the same type as x. The special value `havoc` denotes an arbitrary value of that type. Phi commands use the LLVM-style predecessor list, with `:=` retained as the assignment marker.

Assumptions may mention arbitrary bool expressions. Assertions use a purified condition: the assertion argument is a bool register, not an arbitrary expression.

For example:

```
x := havoc
y := havoc
z := havoc
c := x < y
assume y < z
assert c
```

Thus `assert` conditions are named, while `assume` conditions need not be.

2.4 Blocks and Programs

A program is a collection of basic blocks connected by terminators.

```
block ::= B:
            cmd*
            terminator

terminator ::= halt
             | goto B
             | if c goto B1 else B2
```

Conditional terminators branch on a bool register, so every branch condition is named.

A program has a distinguished entry block and a distinct exit block:

```
program ::= entry = B_entry
            exit  = B_exit
            block*
```

The exit block is the normal target for completed executions; `halt` stops execution at the current block.

2.5 Example

```
entry:
  M := havoc
  i := havoc
  limit := havoc
  x := M[i]
  y := x + 1
  M2 := M[i := y]
  c := y <= limit
  if c goto ok else bad

ok:
```

```
  assert c
  goto exit

bad:
  assume not c
  halt

exit:
  halt
```

The conditional branch is the terminator of `entry`; the assertion in `ok` refers to the named bool register `c`.

2.6 Semantic Intuition

The language follows the expected operational semantics. Assignments update the destination register. A havoc assignment chooses an arbitrary value of the destination type. A load reads a map at an integer address; a store returns a new map that agrees with the old map except at the updated address.

`assume b` keeps only executions where b is true.

`assert c` fails when the named bool register c is false. VC generation is oriented toward this failure condition: the generated formula is satisfiable exactly when such a failure execution exists.

A conditional terminator `if c goto B1 else B2` transfers control to B1 when c is true and to B2 when c is false.

Phi commands select the value associated with the predecessor block from which control entered the current block:

```
join:
  x := phi [left: x_left, right: x_right]
  goto exit
```

If execution reaches `join` from `left`, then `x` receives `x_left`; if it reaches `join` from `right`, then `x` receives `x_right`.

3 Well-Formed Programs

The grammar admits more programs than the VC generators are meant to consume. This section records the structural restrictions used by the two main VC generation styles.

3.1 CFG Vocabulary

The control-flow graph, or CFG, has one node per basic block. There is an edge $b \to b'$ when the terminator of b may transfer control to b'.

The successors of b are the blocks b' such that $b \to b'$. The predecessors of b are the blocks p such that $p \to b$.

An entry-to-exit path is a CFG path that starts at the distinguished entry block and ends at the distinguished exit block.

3.2 SESE

A program is in single-entry single-exit form when:

- it has distinguished, distinct `entry` and `exit` blocks,
- execution starts only at `entry`,
- normal completed executions end at `exit`,
- every block lies on some entry-to-exit path.

The last condition rules out unreachable blocks and dead regions that cannot contribute to an entry-to-exit execution.

3.3 SSA

In static single assignment form:

- every register is assigned at most once,
- phi assignments appear as a contiguous prefix of a basic block,
- a phi assignment has one incoming value for each predecessor of its block.

SSA names merge values explicitly at join blocks:

```
left:
  x_left := 1
  goto join

right:
  x_right := 2
  goto join

join:
  x := phi [left: x_left, right: x_right]
  goto exit
```

3.4 DSA

In dynamic single assignment form, definitions are classified as static or dynamic.

A definition is static when its left-hand side is assigned once in the program. A definition is dynamic when the same left-hand side is assigned in multiple sibling predecessor blocks. Dynamic assignments form a contiguous suffix of the predecessor blocks, immediately before their terminators.

The DSA version of the previous SSA example pushes the phi assignment into the predecessors:

```
left:
  x_left := 1
  x := x_left
  goto join

right:
  x_right := 2
  x := x_right
  goto join
```

```
join:
  goto exit
```

Here the two assignments to `x` are dynamic definitions. They occur in sibling predecessors of `join`, and each assignment is at the end of its block. DSA can also be viewed as placing these assignments on the incoming edges to `join`.

3.5 SSA and DSA Conversion

The conversion from SSA to DSA replaces each phi block:

```
join:
  x := phi [p1: v1, p2: v2, ...]
```

with assignments inserted at the end of the corresponding predecessor blocks:

```
p1:
  ...
  x := v1
  goto join

p2:
  ...
  x := v2
  goto join

join:
  ...
```

The reverse conversion collects sibling dynamic assignments into a phi node at the successor block.

3.6 SASA

A program is in single-assume single-assert form when:

- it is SESE,
- it has exactly one `assume` command and exactly one `assert` command,
- both commands appear in the exit block as:

```
exit:
  assume pre
  assert post
  halt
```

Here `pre` is any bool expression, while `post` is a bool register. There are no other commands in the exit block.

Every program can be reduced to SASA form by accumulating assumptions into a single precondition and assertions into a single postcondition. Informally, `pre` is the conjunction of the assumptions required along an entry-to-exit execution, and `post` summarizes the assertion obligations for that

execution. Earlier assumes and asserts are replaced by ordinary control/data constraints whose effect is reflected in `pre` and `post`.

3.7 Safety

A program is safe when every entry-to-exit execution that satisfies all assumes also satisfies all asserts.

For a SASA program, this becomes:

$$\forall \text{ entry-to-exit executions } \xi.\ \mathrm{pre}(\xi) \Rightarrow \mathrm{post}(\xi)$$

The VC is oriented toward the negation of safety: it asks whether there exists an entry-to-exit execution satisfying `pre` and falsifying `post`.

4 VC Generation Algorithms

VCGen Problem. Given a `ttac` program P in SASA form, $\mathrm{VCGen}(P)$ is a formula Φ such that Φ is satisfiable iff P is unsafe.

For a SASA program, this can be read as an execution query:

$$\exists \xi.\ \mathrm{Path}(\xi) \wedge \mathrm{pre}(\xi) \wedge \neg\ \mathrm{post}(\xi)$$

Here ξ ranges over entry-to-exit executions. The predicate Path captures the program semantics: control flow, assignments, havoc choices, map updates, and branch decisions. The query is satisfiable exactly when there is an execution that satisfies the accumulated precondition and falsifies the final assertion condition.

Different VC generation algorithms choose different ways to represent Path. We consider three styles:

- VCGen_SeaHorn: explicit control representation via block variables with side-conditions, naturally in SSA.
- VCGen_Boogie: weakest-precondition style formulas, naturally over DSA.
- VCGen_SeaBMC: data-flow driven with control compiled into data.

The algorithms should be compared by the shape of the formula they generate, the program form they prefer, and the solver behavior they induce.

4.1 SeaHorn-Style VC Generation

VCGen_SeaHorn keeps control flow explicit by introducing one Boolean variable per basic block, then constraining those variables so they describe an entry-to-exit path.

4.1.1 Basics

The generated formula has the shape:

$$\mathrm{VCGen\_SeaHorn}(P) = \mathrm{CFG} \wedge \mathrm{DEF} \wedge \mathrm{JUMPS} \wedge \mathrm{PHI} \wedge \mathrm{ERROR} \tag{1}$$

Each conjunct has a separate role:

- `CFG` selects a feasible path through the control-flow graph.
- `DEF` gives semantics to static assignments.
- `JUMPS` enforces branch conditions on selected control-flow edges.
- `PHI` gives semantics to phi assignments at joins.
- `ERROR` requires the selected path to violate the final assertion.

CFG
For each basic block $i \in \mathrm{BB}(P)$, introduce a Boolean block variable: bb_i.

The CFG constraints require the entry block, require the exit block, and require every selected non-entry block to have a selected predecessor:

$$\begin{aligned} \mathrm{CFG} = \; & \mathtt{bb}_{\mathrm{entry}} \\ & \wedge \forall i \neq \mathrm{entry}.\mathtt{bb}_i \Rightarrow \vee_{j \in \mathrm{pred}(i)} \mathtt{bb}_j \\ & \wedge \mathtt{bb}_{\mathrm{exit}} \end{aligned} \tag{2}$$

The middle constraint is generated for each non-entry block i. Concrete encodings may add equivalent strengthening constraints, such as edge variables or at-most-one constraints, but the basic purpose is the same: the selected block variables describe a path from entry to exit.

DEF
Static assignments are guarded by the block in which they appear. For a block i, collect its static assignments:

```
x1 := e1
x2 := e2
...
xk := ek
```

and emit:

$$\mathrm{DEF}_i = \mathtt{bb}_i \Rightarrow \wedge_{m=1}^{k} (x_m = e_m) \tag{3}$$

Assignments become equalities in the formula. Havoc assignments do not produce a defining equality; they leave the assigned variable unconstrained except for its type.

JUMPS
Branch conditions are enforced only when the corresponding edge is selected. For a conditional terminator:

```
if c goto t else f
```

the selected true edge implies c, and the selected false edge implies $\neg c$:

$$\begin{aligned} \mathrm{JUMPS}_{i,t} &= (\mathtt{bb}_i \wedge \mathtt{bb}_t) \Rightarrow c \\ \mathrm{JUMPS}_{i,f} &= \left(\mathtt{bb}_i \wedge \mathtt{bb}_f\right) \Rightarrow \neg c \end{aligned} \tag{4}$$

Unconditional edges have no additional jump condition.

PHI
Phi assignments are also edge-sensitive. If block j starts with:

```
x := phi [i: x_i, k: x_k]
```

then selecting edge $i \to j$ selects the corresponding incoming value:

$$\begin{aligned} \mathrm{PHI}_{i,j} &= \left(\mathtt{bb}_i \wedge \mathtt{bb}_j\right) \Rightarrow x = x_i \\ \mathrm{PHI}_{k,j} &= \left(\mathtt{bb}_k \wedge \mathtt{bb}_j\right) \Rightarrow x = x_k \end{aligned} \tag{5}$$

This is the SSA version of the DSA idea that dynamic assignments live on predecessor edges.

ERROR
For a SASA exit block:

```
exit:
  assume pre
  assert post
  halt
```

the error condition requires the selected exit state to satisfy `pre` and falsify `post`:

$$\mathrm{ERROR} = \mathtt{bb}_{\mathrm{exit}} \Rightarrow \mathrm{pre} \wedge \neg\ \mathrm{post} \tag{6}$$

Together with the explicit constraint $\mathtt{bb}_{\mathrm{exit}}$, this asks for a complete entry-to-exit execution that reaches the assertion failure condition.

Example 1: Basic Diamond Encoding
Consider:

```
entry:
  x := havoc
  y := havoc
  c := x < y
  if c goto left else right

left:
  a_left := x + 1
  goto join

right:
  a_right := y + 1
  goto join

join:
  a := phi [left: a_left, right: a_right]
  ok := a > 0
  goto exit

exit:
  assume true
  assert ok
  halt
```

The VCGen_SeaHorn formula contains:

$$
\begin{aligned}
\textbf{CFG} &= \mathtt{bb}_{\text{entry}} \\
&\quad \wedge \left(\mathtt{bb}_{\text{left}} \Rightarrow \mathtt{bb}_{\text{entry}}\right) \\
&\quad \wedge \left(\mathtt{bb}_{\text{right}} \Rightarrow \mathtt{bb}_{\text{entry}}\right) \\
&\quad \wedge \left(\mathtt{bb}_{\text{join}} \Rightarrow \mathtt{bb}_{\text{left}} \vee \mathtt{bb}_{\text{right}}\right) \\
&\quad \wedge \left(\mathtt{bb}_{\text{exit}} \Rightarrow \mathtt{bb}_{\text{join}}\right) \\
&\quad \wedge \mathtt{bb}_{\text{exit}} \\
\textbf{DEF} &= \left(\mathtt{bb}_{\text{entry}} \Rightarrow c = (x < y)\right) \\
&\quad \wedge \left(\mathtt{bb}_{\text{left}} \Rightarrow a_{\text{left}} = x + 1\right) \\
&\quad \wedge \left(\mathtt{bb}_{\text{right}} \Rightarrow a_{\text{right}} = y + 1\right) \\
&\quad \wedge \left(\mathtt{bb}_{\text{join}} \Rightarrow \text{ok} = (a > 0)\right) \\
\textbf{JUMPS} &= \left(\left(\mathtt{bb}_{\text{entry}} \wedge \mathtt{bb}_{\text{left}}\right) \Rightarrow c\right) \\
&\quad \wedge \left(\left(\mathtt{bb}_{\text{entry}} \wedge \mathtt{bb}_{\text{right}}\right) \Rightarrow \neg c\right) \\
\textbf{PHI} &= \left(\left(\mathtt{bb}_{\text{left}} \wedge \mathtt{bb}_{\text{join}}\right) \Rightarrow a = a_{\text{left}}\right) \\
&\quad \wedge \left(\left(\mathtt{bb}_{\text{right}} \wedge \mathtt{bb}_{\text{join}}\right) \Rightarrow a = a_{\text{right}}\right) \\
\textbf{ERROR} &= \mathtt{bb}_{\text{exit}} \Rightarrow \top \wedge \neg\ \text{ok}
\end{aligned}
$$

The satisfying models of this formula are precisely candidate unsafe executions: selected blocks form an entry-to-exit path, assignments and edge conditions agree with that path, and the exit assertion is false.

4.1.2 Optimizations

The basic encoding in Equation 1 fixes the semantic pieces of VCGen_SeaHorn, but not their exact SMT shape. The implementation exposes several equivalent or strengthening variants whose purpose is to help the SMT solver propagate path choices, simplify definitions, or avoid irrelevant terms.

CFG Variants

The CFG component can be encoded in several ways.

- backward, using predecessors: $\mathtt{bb}_i \Rightarrow \vee_{j \in \text{ pred}(i)}\ \mathtt{bb}_j$;
- forward, using successors: $\mathtt{bb}_i \Rightarrow \vee_{j \in \text{ succ}(i)}\ \mathtt{bb}_j$;
- with explicit edge variables $e_{i,j}$;
- forward plus backward edges to immediate dominators: $\mathtt{bb}_i \Rightarrow \mathtt{bb}_{\text{idom}(i)}$;
- with exclusivity constraints for incompatible choices, such as at-most-one selected successor or at-most-one selected predecessor.

For a diamond:

```
entry:
  c := havoc
  if c goto left else right

left:
```

```
  goto join

right:
  goto join
```

backward reachability for `join` says:

$$\mathtt{bb}_{\text{join}} \Rightarrow \mathtt{bb}_{\text{left}} \vee \mathtt{bb}_{\text{right}}$$

whereas a forward encoding starts from `entry`:

$$\mathtt{bb}_{\text{entry}} \Rightarrow \mathtt{bb}_{\text{left}} \vee \mathtt{bb}_{\text{right}}$$

An edge-variable encoding can make the selected edge explicit:

$$e_{\text{entry,left}} \Rightarrow \mathtt{bb}_{\text{entry}} \wedge \mathtt{bb}_{\text{left}} \wedge c$$
$$e_{\text{entry,right}} \Rightarrow \mathtt{bb}_{\text{entry}} \wedge \mathtt{bb}_{\text{right}} \wedge \neg c$$

Exclusivity constraints rule out spurious models that select both branches:

$$\neg\left(\mathtt{bb}_{\text{left}} \wedge \mathtt{bb}_{\text{right}}\right)$$

These constraints are logically redundant in an ideal path semantics, but they can give the solver shorter Boolean-propagation paths.

Unguarded Static Definitions
Static definitions can either be guarded by their defining block or emitted as top-level equations:

$$\mathtt{bb}_i \Rightarrow x = y + 1$$

$$x = y + 1$$

The unguarded form is sound for static SSA-style definitions when the symbol's uses are already controlled by reachability constraints. If x is unused, the equation is simply irrelevant. If x is used, exposing the equality at top level lets the SMT solver eliminate the intermediate name early.

For example:

$$x = y + 1 \wedge z = x + 2$$

can simplify by substitution to:

$$z = y + 3$$

This is the motivation for leaving definition guards optional: guarded definitions preserve a direct path-scoped reading, while unguarded definitions make algebraic simplification easier.

Phi as ITE
Phi constraints can be emitted as edge-specific implications, as in Equation 5:

$$\left(\mathtt{bb}_i \wedge \mathtt{bb}_j\right) \Rightarrow x = x_i$$

$$\left(\mathtt{bb}_k \wedge \mathtt{bb}_j\right) \Rightarrow x = x_k$$

Alternatively, the merge can be expressed as one explicit defining equation:

$$x = \mathrm{ite}(\mathtt{bb}_i, x_i, x_k)$$

The ITE form makes the definition of x syntactically explicit. The solver can then decide whether to keep the ITE, pull it through a use site, or simplify it after Boolean propagation has fixed the selected predecessor.

For example, after learning $\mathtt{bb}_i$, the equation:

$$x = \mathrm{ite}(\mathtt{bb}_i, x_i, x_k)$$

collapses to:

$$x = x_i$$

Example 2: Optimized Diamond Encoding
Using the same diamond program from Example 1, choose:

- forward CFG constraints,
- unguarded static definitions,
- ITE encoding for the phi node.

Then the formula components become:

$$
\begin{aligned}
\textbf{CFG} &= \mathtt{bb}_{\text{entry}} \\
&\quad \wedge \left(\mathtt{bb}_{\text{entry}} \Rightarrow \mathtt{bb}_{\text{left}} \vee \mathtt{bb}_{\text{right}}\right) \\
&\quad \wedge \left(\mathtt{bb}_{\text{left}} \Rightarrow \mathtt{bb}_{\text{join}}\right) \\
&\quad \wedge \left(\mathtt{bb}_{\text{right}} \Rightarrow \mathtt{bb}_{\text{join}}\right) \\
&\quad \wedge \left(\mathtt{bb}_{\text{join}} \Rightarrow \mathtt{bb}_{\text{exit}}\right) \\
&\quad \wedge \mathtt{bb}_{\text{exit}} \\
\textbf{DEF} &= c = (x < y) \\
&\quad \wedge a_{\text{left}} = x + 1 \\
&\quad \wedge a_{\text{right}} = y + 1 \\
&\quad \wedge a = \mathrm{ite}\left(\mathtt{bb}_{\text{left}}, a_{\text{left}}, a_{\text{right}}\right) \\
&\quad \wedge \mathrm{ok} = (a > 0) \\
\textbf{JUMPS} &= \left(\left(\mathtt{bb}_{\text{entry}} \wedge \mathtt{bb}_{\text{left}}\right) \Rightarrow c\right) \\
&\quad \wedge \left(\left(\mathtt{bb}_{\text{entry}} \wedge \mathtt{bb}_{\text{right}}\right) \Rightarrow \neg c\right) \\
\textbf{PHI} &= \top \\
\textbf{ERROR} &= \mathtt{bb}_{\text{exit}} \Rightarrow \top \wedge \neg\, \mathrm{ok}
\end{aligned}
$$

The CFG constraints now push reachability forward from each selected block to a successor. Static definitions are visible as top-level equations. The phi component is empty because the merge is represented directly by the ITE definition of a.

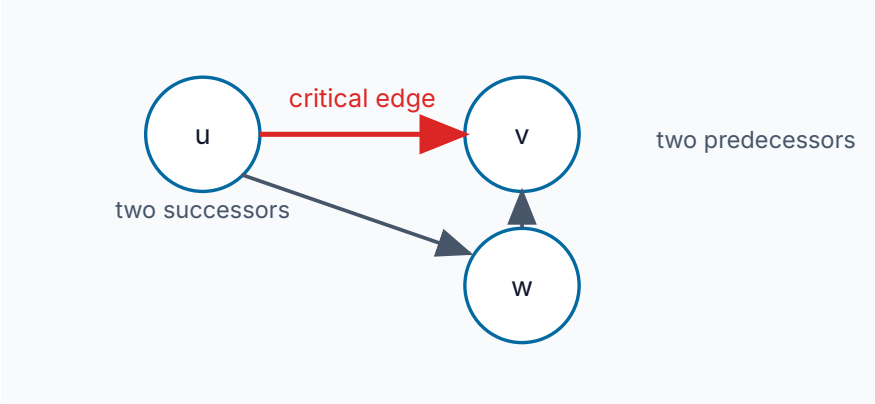
4.1.3 Common Soundness Pitfalls

The SeaHorn-style encoding is compact, but small changes can silently change the meaning of the generated formula. The following mistakes have appeared in the history of this encoder family.

Critical Edges

An edge $u \to v$ is critical when u has more than one successor and v has more than one predecessor:

$$|\mathrm{succ}(u)| > 1 \wedge |\mathrm{pred}(v)| > 1$$



The problem is that block variables do not identify which incoming edge was taken. On a critical edge, $\texttt{bb}_u \wedge \texttt{bb}_v$ may be true even when execution reaches v through a different predecessor.

For example:

```
entry:
  c := havoc
  if c goto join else mid

mid:
  goto join

join:
  ok := phi [entry: true, mid: false]
  goto exit

exit:
  assume true
  assert ok
  halt
```

The edge entry $\rightarrow$ join is critical: `entry` has two successors and `join` has two predecessors. The real program is unsafe: when c is false, execution goes through `mid`, reaches `join`, sets `ok := false`, and fails the assert.

But if the encoder represents the true edge condition as:

$$\left(\texttt{bb}_{\text{entry}} \wedge \texttt{bb}_{\text{join}}\right) \Rightarrow c$$

then the feasible path `entry -> mid -> join` is incorrectly rejected, because both $\texttt{bb}_{\text{entry}}$ and $\texttt{bb}_{\text{join}}$ are true on that path while c is false. The encoder can miss the bug.

The standard repair is to split critical edges by inserting a landing block:

```
entry:
  c := havoc
  if c goto entry_to_join else mid

entry_to_join:
  goto join

mid:
  goto join
```

Now the edge-specific condition is attached to the non-critical edge entry $\rightarrow$ entry_to_join rather than to entry $\rightarrow$ join.

Effect: this can cause **missing paths**. A real entry-to-exit execution is ruled out by constraints that were meant for a different edge.

Missing Predecessor Exclusivity for ITE Merges
When a phi merge is encoded as implications, selecting two predecessors often creates an immediate contradiction:

$$\left(\mathtt{bb}_i \wedge \mathtt{bb}_j\right) \Rightarrow x = x_i$$

$$\left(\mathtt{bb}_k \wedge \mathtt{bb}_j\right) \Rightarrow x = x_k$$

If $x_i \neq x_k$ and both predecessors are selected, the equalities conflict. With an ITE merge, however, the formula contains only one equality:

$$x = \mathrm{ite}(\mathtt{bb}_i, x_i, x_k)$$

If both $\mathtt{bb}_i$ and $\mathtt{bb}_k$ are true, the ITE simply chooses one arm. A model can therefore satisfy the CFG part using an impossible merge state, and different merged variables may be read from different incoming edges. The result is a spurious counterexample: the formula describes a mixed state that no execution can produce.

For example:

```
entry:
  c := havoc
  if c goto left else right

left:
  x_left := true
  p_left := true
  goto join

right:
  x_right := false
  p_right := false
  goto join

join:
  x := phi [left: x_left, right: x_right]
  p := phi [left: p_left, right: p_right]
  ok := x == p
  goto exit

exit:
  assume true
  assert ok
  halt
```

The program is safe. On the left path, both x and p are true. On the right path, both are false. Now suppose the two phi nodes are emitted as independent ITE definitions, and their incoming cases appear in different orders:

$$x = \mathrm{ite}\big(\mathtt{bb}_{\mathrm{left}}, x_{\mathrm{left}}, x_{\mathrm{right}}\big)$$

$$p = \mathrm{ite}\big(\mathtt{bb}_{\mathrm{right}}, p_{\mathrm{right}}, p_{\mathrm{left}}\big)$$

If the CFG constraints allow both $\mathtt{bb}_{\mathrm{left}}$ and $\mathtt{bb}_{\mathrm{right}}$, then the first ITE can choose the left value while the second chooses the right value. The formula can set $x = \mathrm{true}$ and $p = \mathrm{false}$, making ok false even though no entry-to-exit execution reaches such a state.

The repair is an at-most-one predecessor constraint at the merge:

$$\neg\big(\mathtt{bb}_{\mathrm{left}} \wedge \mathtt{bb}_{\mathrm{right}}\big)$$

Equivalently, a CFG encoding that uses incoming edge variables must enforce that at most one incoming edge to the merge fires.

Effect: this can cause **infeasible paths**. The formula admits a merge state that does not correspond to any single predecessor edge.

Unguarded Path-Local Facts
Unguarded definitions are useful when the RHS is total and the definition is static. They become dangerous when the RHS carries a path-local axiom or a partial-operation side condition.

Consider a pseudo-operation `narrow64` whose encoding treats it as identity but also adds a 64-bit range fact for its result:

```
entry:
  x := havoc
  small := x < 2^64
  if small goto narrow_block else exit

narrow_block:
  y := narrow64(x)
  goto exit

exit:
  assume not small
  assert false
  halt
```

The program is unsafe: choose $x \geq 2^{64}$. Then `small` is false, execution goes directly to `exit`, the assumption `not small` holds, and `assert false` fails.

If `y := narrow64(x)` is emitted globally as:

$$y = x \wedge 0 \leq y \wedge y \leq 2^{64} - 1$$

then the encoder has constrained x even on the path that never reaches `narrow_block`. On the failing path, `not small` requires $x \geq 2^{64}$, while the global range fact requires $x \leq 2^{64} - 1$. The bad encoding therefore makes the failing path inconsistent.

The guarded form keeps the range fact local:

$$\mathtt{bb}_{\text{narrow\_block}} \Rightarrow (y = x \wedge 0 \leq y \wedge y \leq 2^{64} - 1)$$

The same issue occurs for operator axioms whose validity depends on where the operator call appears. Axioms for such calls must be guarded by the triggering block, or otherwise justified as globally safe.

Effect: this can cause **missing paths**. Facts from an unvisited block constrain an execution that should not see those facts.

4.2 Boogie-Style VCGen

The Boogie-style VC generator interprets verification as a structured weakest-precondition computation over a CFG. We present a simplified variant tailored to loop-free programs in DSA form.

For this section, assume:

- the input program is in DSA form;
- there are no phi nodes;
- dynamic DSA definitions that depend on the predecessor edge are attached to that edge.

For each basic block i, introduce a Boolean variable ok_i. Intuitively, ok_i means: starting execution at block i is safe. The final verification condition asks for a counterexample by requiring the entry block not to be safe:

$$\neg \mathrm{ok}_{\text{entry}} \tag{7}$$

4.2.1 Block Equations

For each block i, define ok_i by one equation. Let:

- defs_i be the conjunction of the static definitions and assumptions in block i;
- $\mathrm{asserts}_i$ be the conjunction of the assertion predicates in block i, or $\top$ if the block has no assertions;
- $\mathrm{cond}_{i,j}$ be the edge condition for edge $i \to j$; for an unconditional edge, $\mathrm{cond}_{i,j} = \top$;
- $\mathrm{dsa}_{i,j}$ be the conjunction of DSA edge definitions for edge $i \to j$, or $\top$ if the edge has no such definitions.

Then:

$$\mathrm{ok}_i \Leftrightarrow \Big(\mathrm{defs}_i \Rightarrow \Big(\mathrm{asserts}_i \wedge \wedge_{j \in \mathrm{succ}(i)} \big((\mathrm{cond}_{i,j} \wedge \mathrm{dsa}_{i,j}) \Rightarrow \mathrm{ok}_j\big)\Big)\Big) \tag{8}$$

For a terminal block, the successor conjunction is $\top$, so the equation reduces to:

$$\mathrm{ok}_i \Leftrightarrow (\mathrm{defs}_i \Rightarrow \mathrm{asserts}_i) \tag{9}$$

The shape is backward: a block is safe when its local facts imply that every feasible successor state is safe. Assumptions appear on the left of the implication, so an infeasible block is safe vacuously. Assertions appear on the right, so a reachable false assertion makes the corresponding ok_i false.

Havoc assignments do not contribute equalities to defs_i; they leave their destination variables unconstrained. Static assignments contribute ordinary equalities. Edge-sensitive DSA definitions contribute to $\mathrm{dsa}_{i,j}$.

4.2.2 Example 1: Basic Diamond Encoding

Consider the diamond from Example 1, written in pure DSA form by attaching the merge assignment for `a` to the incoming edges of `join`:

```
entry:
  x := havoc
  y := havoc
  c := x < y
  if c goto left else right

left:
  a_left := x + 1
  a := a_left
  goto join

right:
  a_right := y + 1
  a := a_right
  goto join

join:
  ok := a > 0
  goto exit

exit:
  assume true
  assert ok
  halt
```

The DSA edge definitions are:

$$\mathrm{dsa}_{\mathrm{left},\mathrm{join}} = (a = a_{\mathrm{left}})$$
$$\mathrm{dsa}_{\mathrm{right},\mathrm{join}} = (a = a_{\mathrm{right}})$$

The Boogie-style VC is:

$$
\begin{aligned}
\mathbf{OK}_{\text{entry}} &: \mathrm{ok}_{\text{entry}} \Leftrightarrow \big(c = (x < y) \Rightarrow \big((c \Rightarrow \mathrm{ok}_{\text{left}}) \wedge (\neg c \Rightarrow \mathrm{ok}_{\text{right}})\big)\big) \\
\mathbf{OK}_{\text{left}} &: \mathrm{ok}_{\text{left}} \Leftrightarrow \big(a_{\text{left}} = x + 1 \Rightarrow \big((a = a_{\text{left}}) \Rightarrow \mathrm{ok}_{\text{join}}\big)\big) \\
\mathbf{OK}_{\text{right}} &: \mathrm{ok}_{\text{right}} \Leftrightarrow \big(a_{\text{right}} = y + 1 \Rightarrow \big((a = a_{\text{right}}) \Rightarrow \mathrm{ok}_{\text{join}}\big)\big) \\
\mathbf{OK}_{\text{join}} &: \mathrm{ok}_{\text{join}} \Leftrightarrow (\mathrm{ok} = (a > 0) \Rightarrow \mathrm{ok}_{\text{exit}}) \\
\mathbf{OK}_{\text{exit}} &: \mathrm{ok}_{\text{exit}} \Leftrightarrow \mathrm{ok} \\
\mathbf{DISCHARGE} &: \neg \mathrm{ok}_{\text{entry}}
\end{aligned}
$$

The model of the VC chooses values for the havoc variables and for the edge-defined merge variable a. The equation for `entry` says that the unsafe witness must follow a branch whose successor is unsafe; the equations for `left` and `right` add the corresponding DSA edge definition before checking `join`.

4.2.3 Example 2: Optimized Diamond Encoding

Using the optimized diamond from Example 2, the merge can instead be represented as a static ITE at `join`:

```
entry:
  x := havoc
  y := havoc
  c := x < y
  if c goto left else right

left:
  a_left := x + 1
  goto join

right:
  a_right := y + 1
  goto join

join:
  a := ite(c, a_left, a_right)
  ok := a > 0
  goto exit

exit:
  assume true
  assert ok
  halt
```

Now there are no edge definitions for `join`; the merge is part of $\mathrm{defs}_{\text{join}}$:

$$
\begin{aligned}
\textbf{OK}_{\text{entry}} &: \mathrm{ok}_{\text{entry}} \Leftrightarrow \big(c = (x < y) \Rightarrow \big((c \Rightarrow \mathrm{ok}_{\text{left}}) \wedge (\neg c \Rightarrow \mathrm{ok}_{\text{right}})\big)\big) \\
\textbf{OK}_{\text{left}} &: \mathrm{ok}_{\text{left}} \Leftrightarrow \big(a_{\text{left}} = x + 1 \Rightarrow \mathrm{ok}_{\text{join}}\big) \\
\textbf{OK}_{\text{right}} &: \mathrm{ok}_{\text{right}} \Leftrightarrow \big(a_{\text{right}} = y + 1 \Rightarrow \mathrm{ok}_{\text{join}}\big) \\
\textbf{OK}_{\text{join}} &: \mathrm{ok}_{\text{join}} \Leftrightarrow \big(\big(a = \mathrm{ite}\big(c, a_{\text{left}}, a_{\text{right}}\big) \wedge \mathrm{ok} = (a > 0)\big) \Rightarrow \mathrm{ok}_{\text{exit}}\big) \\
\textbf{OK}_{\text{exit}} &: \mathrm{ok}_{\text{exit}} \Leftrightarrow \mathrm{ok} \\
\textbf{DISCHARGE} &: \neg \mathrm{ok}_{\text{entry}}
\end{aligned}
$$

This version has the same backward shape, but the predecessor-sensitive merge has been compiled into the local definition of `a` at `join`.

4.2.4 Tradeoffs

Advantages:

- No special CFG constraints are needed. The CFG structure is compiled into the recursive ok_i equations.
- Arbitrarily many assertions are easy to support. Each assertion contributes another local consequent to its block equation.

Disadvantages:

- The formula is structurally more complex than a flat path formula.
- Cross-block simplification is harder, because facts are nested under the block equations rather than exposed as top-level constraints.

4.2.5 Extension

This style of encoding can also support a mixture of phi nodes and DSA assignments. The DSA assignments remain part of the local block premises or dynamic edge premises, while phi assignments are compiled into static definitions using ITE expressions over basic-block variables.

In this extension, the Boogie-style block equations are combined with ordinary CFG constraints. The CFG variables serve two purposes:

- they define the reachability terms used in the phi ITEs;
- they enforce predecessor exclusivity at merge blocks.

The exclusivity condition is important. If a phi node is encoded as:

$$
\textbf{PHI}_{\text{a}} : a = \mathrm{ite}\big(\mathtt{bb}_{\text{left}}, a_{\text{left}}, a_{\text{right}}\big)
$$

then the formula must ensure that at most one incoming predecessor feeding the ITE is selected. Otherwise, two predecessor blocks can be true at once and the ITE will silently choose one arm, creating a mixed state that does not correspond to any real execution.

This requires:

- no critical edges, so predecessor block variables identify incoming edges to the merge;
- exclusivity constraints such as $\neg\big(\mathtt{bb}_{\text{left}} \wedge \mathtt{bb}_{\text{right}}\big)$ for the predecessors of a phi merge.

For the running diamond:

```
entry:
  x := havoc
  y := havoc
  c := x < y
  if c goto left else right

left:
  a_left := x + 1
  goto join

right:
  a_right := y + 1
  goto join

join:
  a := phi [left: a_left, right: a_right]
  ok := a > 0
  goto exit

exit:
  assume true
  assert ok
  halt
```

the phi node can be compiled into:

$$\mathbf{PHI}_{\mathrm{a}} : a = \mathrm{ite}\big(\mathtt{bb}_{\mathrm{left}}, a_{\mathrm{left}}, a_{\mathrm{right}}\big)$$

and the CFG side conditions include:

$$\begin{aligned}
\mathbf{CFG}_{\mathrm{entry}} &: \mathtt{bb}_{\mathrm{entry}} \\
\mathbf{CFG}_{\mathrm{left}} &: \mathtt{bb}_{\mathrm{left}} \Rightarrow \mathtt{bb}_{\mathrm{entry}} \wedge c \\
\mathbf{CFG}_{\mathrm{right}} &: \mathtt{bb}_{\mathrm{right}} \Rightarrow \mathtt{bb}_{\mathrm{entry}} \wedge \neg c \\
\mathbf{CFG}_{\mathrm{join}} &: \mathtt{bb}_{\mathrm{join}} \Rightarrow \mathtt{bb}_{\mathrm{left}} \vee \mathtt{bb}_{\mathrm{right}} \\
\mathbf{CFG}_{\mathrm{excl}} &: \neg\big(\mathtt{bb}_{\mathrm{left}} \wedge \mathtt{bb}_{\mathrm{right}}\big)
\end{aligned}$$

The corresponding block equations keep the same backward shape:

$$
\begin{aligned}
\textbf{OK}_{\text{entry}} &: \mathrm{ok}_{\text{entry}} \Leftrightarrow \big(c = (x < y) \Rightarrow \big((c \Rightarrow \mathrm{ok}_{\text{left}}) \wedge (\neg c \Rightarrow \mathrm{ok}_{\text{right}})\big)\big) \\
\textbf{OK}_{\text{left}} &: \mathrm{ok}_{\text{left}} \Leftrightarrow \big(a_{\text{left}} = x + 1 \Rightarrow \mathrm{ok}_{\text{join}}\big) \\
\textbf{OK}_{\text{right}} &: \mathrm{ok}_{\text{right}} \Leftrightarrow \big(a_{\text{right}} = y + 1 \Rightarrow \mathrm{ok}_{\text{join}}\big) \\
\textbf{OK}_{\text{join}} &: \mathrm{ok}_{\text{join}} \Leftrightarrow \big(\big(a = \mathrm{ite}\big(\mathtt{bb}_{\text{left}}, a_{\text{left}}, a_{\text{right}}\big) \wedge \mathrm{ok} = (a > 0)\big) \Rightarrow \mathrm{ok}_{\text{exit}}\big) \\
\textbf{OK}_{\text{exit}} &: \mathrm{ok}_{\text{exit}} \Leftrightarrow \mathrm{ok} \\
\textbf{DISCHARGE} &: \neg \mathrm{ok}_{\text{entry}}
\end{aligned}
$$

The extra CFG constraints are not used to select a path directly in the Boogie-style recurrence; they make the block variables used by phi ITEs consistent enough that the ITE definitions denote a single incoming edge.

4.3 SeaBMC-Style VCGen

Our goal is a VCGen algorithm that does not need separate CFG constraints, as in the Boogie-style encoding, but remains direct, as in the SeaHorn-style encoding.

The intuition is to reduce all control-dependence into data-dependence, so that program statements can execute in arbitrary order, and the assertion and assumption use only the values they require.

Assume that P is in SSA, SESE, and SASA form: every variable is assigned only once, joins are represented by phi nodes, all paths go from `entry` to `exit`, and the designated exit block contains the single assumption and assertion.

Direct phi encoding needs CFG information. The trick is to rewrite the input program so phi nodes disappear before VC generation. In compilers, this form is known as **Gated SSA**. A phi node is replaced by a gamma node guarded by Boolean program expressions rather than by predecessor block names.

4.3.1 Gated SSA

A gamma node is a value-level merge:

$$x := \gamma(g, x_t, x_f)$$

It means that x takes x_t when the guard g is true, and x_f when the guard is false. Unlike a phi node, a gamma node does not mention predecessor blocks or any auxiliary block-reachability variables. It mentions only ordinary program expressions.

Consider the running diamond in SSA form:

```
entry:
  x := havoc
  y := havoc
  c := x < y
  if c goto left else right

left:
  a_left := x + 1
  goto join
```

```
right:
  a_right := y + 1
  goto join

join:
  a := phi [left: a_left, right: a_right]
  ok := a > 0
  goto exit

exit:
  assume true
  assert ok
  halt
```

In Gated SSA, the phi node is replaced by a gamma node guarded by the branch condition that controls the choice:

```
entry:
  x := havoc
  y := havoc
  c := x < y
  if c goto left else right

left:
  a_left := x + 1
  goto join

right:
  a_right := y + 1
  goto join

join:
  a := gamma(c, a_left, a_right)
  ok := a > 0
  goto exit

exit:
  assume true
  assert ok
  halt
```

Gamma nodes add no new semantics. We use `gamma` only as syntactic sugar for an ITE expression:

$$x := \gamma(g, x_t, x_f) \quad \text{means} \quad x := \text{ite}(g, x_t, x_f)$$

Thus the example above is interpreted as:

```
entry:
  x := havoc
```

```
  y := havoc
  c := x < y
  if c goto left else right

left:
  a_left := x + 1
  goto join

right:
  a_right := y + 1
  goto join

join:
  a := ite(c, a_left, a_right)
  ok := a > 0
  goto exit

exit:
  assume true
  assert ok
  halt
```

After this rewrite, the merge assignment for `a` is an ordinary SSA definition. It no longer needs a CFG predicate to explain which incoming edge was taken.

4.3.2 Gate Construction

The GSSA conversion computes, for each block b, a Boolean expression $\mathrm{gate}(b)$ that is true exactly when execution reaches b, but written in terms of program branch conditions rather than block variables.

We define **control-dependence** using **postdominators**. A block d **postdominates** block b when every path from b to `exit` goes through d. A block b_1 is **control-dependent** on branch block b_2 iff:

- b_2 has a successor s such that b_1 postdominates s;
- b_1 does not postdominate b_2.

Intuitively, after execution takes edge $b_2 \to s$, it must eventually reach b_1. Before the branch at b_2 is resolved, however, reaching b_1 is not forced. This is illustrated in Figure 1.

The **control-dependence graph** has one node for each basic block and an edge $c \to b$ whenever b is control-dependent on branch block c by the definition above. Thus the edge points from the controlling branch block to the block whose execution it controls.

Let $\mathrm{ctrl}(b)$ be the set of branch blocks on which b is control-dependent. A controller $c \in \mathrm{ctrl}(b)$ is therefore a basic block whose terminator is a conditional branch. Define $\mathrm{orient}(c, b)$ as:

- the branch condition of c when b lies under the true successor of c;
- the negated branch condition when b lies under the false successor of c.

In Figure 1, s is the true successor of b_2. Therefore, if the branch condition at b_2 is g, then $\mathrm{orient}(b_2, b_1) = g$. The other successor t is the false successor, so a block controlled by the t branch would use the orientation $\neg g$.

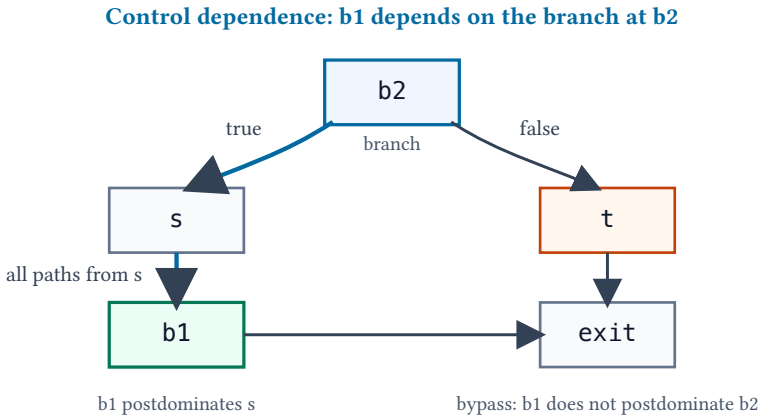


Figure 1: Block b_1 is control-dependent on branch block b_2: the edge $b_2 \to s$ forces a later visit to b_1, while the other branch can bypass b_1.

The gate equations are:

$$\begin{aligned} \text{gate}(\text{entry}) &= \top \\ \text{gate}(b) &= \vee_{c \in \text{ ctrl}(b)} \left(\text{gate}(c) \wedge \text{orient}(c, b)\right) \end{aligned} \tag{10}$$

When $\text{ctrl}(b)$ is empty, $\text{gate}(b) = \top$. In a structured diamond, $\text{gate}(\text{left}) = c$ and $\text{gate}(\text{right}) = \neg c$.

For a phi node in block j:

```
j:
  x := phi [p1: v1, p2: v2, ..., pn: vn]
```

replace it with a nested gamma expression over the predecessor gates:

$$x := \gamma(\text{gate}(p_1), v_1, \gamma(\text{gate}(p_2), v_2, \ldots \gamma(\text{gate}(p_{n-1}), v_{n-1}, v_n)))$$

For a two-predecessor join, this is simply:

$$x := \gamma(\text{gate}(p_1), v_1, v_2)$$

The case order is irrelevant when the gates are mutually exclusive on real executions. For readability, examples use the branch condition directly when it is syntactically equal to the predecessor gate.

4.3.3 Conversion Algorithm

The SSA-to-GSSA conversion is a source-to-source pass.

1. Compute postdominators and control-dependence for the CFG.
2. For every block b that can be used by a merge, compute $\text{gate}(b)$ from Equation 10 in topological order over the control-dependence relation.
3. For every phi node, replace each predecessor label by the predecessor gate.
4. Emit the resulting gamma expression as an ordinary `ite`.
5. Delete the phi node.

After the pass, the program contains only ordinary assignments, havoc commands, the final assumption, and the final assertion. Its VC is the flat conjunction:

$$\mathrm{VCGen\_SeaBMC}(P) = \mathrm{DEF} \wedge \mathrm{ASSUME} \wedge \neg\ \mathrm{ASSERT} \tag{11}$$

where `DEF` is the conjunction of all assignment equalities after phi elimination. Havoc assignments contribute no equality.

For the GSSA diamond above:

$$\begin{aligned}
\mathbf{DEF} &= c = (x < y) \\
&\quad \wedge\ a_{\text{left}} = x + 1 \\
&\quad \wedge\ a_{\text{right}} = y + 1 \\
&\quad \wedge\ a = \mathrm{ite}\big(c, a_{\text{left}}, a_{\text{right}}\big) \\
&\quad \wedge\ \mathrm{ok} = (a > 0) \\
\mathbf{ASSUME} &= \top \\
\mathbf{ASSERT} &= \mathrm{ok} \\
\mathbf{VC} &= \mathbf{DEF} \wedge \top \wedge \neg\ \mathrm{ok}
\end{aligned}$$

The formula has no block variables and no separate CFG constraints. The only remaining trace of control flow is the ITE guard in the definition of `a`.

4.3.4 Materialized Gates

The previous description hides a possible exponential blow-up. The definition of $\mathrm{gate}(b)$ is recursive: it may mention gates of controller blocks, which may mention gates of their own controllers. If every gate expression is substituted textually into every gamma node, shared control-dependence prefixes are copied at each use. A gate DAG can therefore become an exponentially large Boolean expression tree.

Materialized gates avoid this by introducing ordinary Boolean SSA variables for gate predicates. Rather than guarding each gamma by a fully expanded expression, the conversion introduces one named gate for each needed block and lets gammas refer to those names.

The materialized form is:

$$\begin{aligned}
G_{\text{entry}} &:= \top \\
G_b &:= \vee_{c \in\ \mathrm{ctrl}(b)}\ (G_c \wedge \mathrm{orient}(c, b)) \\
x &:= \gamma\big(G_p, v_p, v_q\big)
\end{aligned}$$

Here each G_b is assigned once. The definitions may still form a DAG of control-dependence, but the DAG is represented directly instead of being duplicated into every use site.

Computing Materialized Gates

The materializing conversion is:

1. Compute postdominators and control-dependence.
2. Mark every predecessor block that is mentioned by a phi node. These blocks need gates because the corresponding gamma cases must be guarded.

3. Close the marked set under controllers: if b is marked, mark every $c \in \mathrm{ctrl}(b)$. Repeat to a fixed point.
4. Emit one Boolean SSA definition G_b for every marked block, in topological order over the control-dependence relation.
5. Replace each phi node with a gamma node guarded by the corresponding G_p variables.
6. Emit gamma nodes as ordinary `ite` expressions.

The resulting VC shape is still CFG-free. The CFG is used only by the preprocessing pass that computes gate definitions. After that, the VC contains ordinary assignment equalities, assumptions, and the negated assertion.

Example 1: Shared Gate Structure
Materialized gates help when several merges reuse the same control-dependence structure. Consider a program whose gates have this shape:

$$\begin{aligned}
\mathrm{gate}(a) &= p \\
\mathrm{gate}(b) &= q \\
\mathrm{gate}(c) &= (\mathrm{gate}(a) \wedge r) \vee (\mathrm{gate}(b) \wedge s) \\
\mathrm{gate}(d) &= (\mathrm{gate}(b) \wedge t) \vee (\mathrm{gate}(c) \wedge u) \\
\mathrm{gate}(e) &= (\mathrm{gate}(c) \wedge v) \vee (\mathrm{gate}(d) \wedge w)
\end{aligned}$$

Suppose two phi nodes use the predecessor gates for d and e:

```
join1:
  x := phi [d: x_d, e: x_e]

join2:
  y := phi [d: y_d, e: y_e]
```

The non-thin conversion substitutes the full expressions for $\mathrm{gate}(d)$ and $\mathrm{gate}(e)$ into both gamma nodes:

$$\begin{aligned}
x &:= \gamma(\mathrm{gate}(d), x_d, \gamma(\mathrm{gate}(e), x_e, x_{\mathrm{other}})) \\
y &:= \gamma(\mathrm{gate}(d), y_d, \gamma(\mathrm{gate}(e), y_e, y_{\mathrm{other}}))
\end{aligned}$$

If the gates are then expanded textually, both x and y contain copies of $\mathrm{gate}(e)$, each copy contains a copy of $\mathrm{gate}(d)$ and $\mathrm{gate}(c)$, and the copying continues through the shared ancestors. Adding more joins that use the same gates repeats the whole tree again.

The materialized-gate form shares the gates once:

```
gate_a := p
gate_b := q
gate_c := ite(gate_a, r, false) or ite(gate_b, s, false)
gate_d := ite(gate_b, t, false) or ite(gate_c, u, false)
gate_e := ite(gate_c, v, false) or ite(gate_d, w, false)
```

```
join1:
  x := gamma(gate_d, x_d, gamma(gate_e, x_e, x_other))

join2:
  y := gamma(gate_d, y_d, gamma(gate_e, y_e, y_other))
```

Equivalently, after desugaring gamma:

```
gate_a := p
gate_b := q
gate_c := (gate_a and r) or (gate_b and s)
gate_d := (gate_b and t) or (gate_c and u)
gate_e := (gate_c and v) or (gate_d and w)

join1:
  x := ite(gate_d, x_d, ite(gate_e, x_e, x_other))

join2:
  y := ite(gate_d, y_d, ite(gate_e, y_e, y_other))
```

Now the shared control-dependence structure is linear in the number of gate definitions plus the number of gamma uses.

4.3.5 Thin Gated SSA

Materializing gates prevents repeated expansion, but it does not by itself make the gates small. **Thin Gated SSA** uses only the **direct** control-dependence controllers of a block instead of enumerating every complete path from `entry` to that block.

A block b is **directly control-dependent** on a branch block c when there is an edge $c \to b$ in the control-dependence graph. A block b is control-dependent on c in the broader, transitive sense when c reaches b by a path of one or more control-dependence edges. Thin GSSA uses only the one-edge controllers of b as gamma cases. Controllers that are farther away are not repeated in those cases; they are referenced through the already-materialized gate G_c.

The ITE is over the direct controllers, but the case predicate for a controller c is not just $\mathrm{orient}(c, b)$. It is:

$$K_{c,b} = G_c \wedge \mathrm{orient}(c, b)$$

The factor G_c says that controller block c itself is reached. Without it, the branch condition of an unreachable controller could still choose a case, because ordinary SSA variables are total in the formula. For example, if block `a` is not reached, its branch condition `c2` may still have a model value; switching on `c2` alone could incorrectly select the `a`-side incoming value.

If no direct-controller case $K_{c,b}$ fires, execution does not reach b. For the Boolean reachability gate, the fallback is `false`. For a value gamma in b, the fallback can be `undef`, because the value is unobservable when b is not reached. This distinction is illustrated in Figure 2.

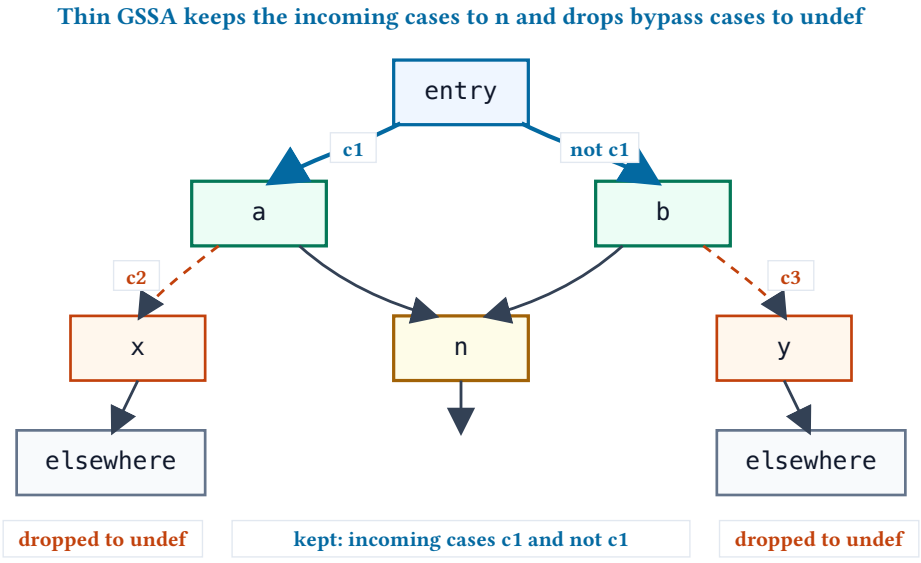


Figure 2: Thin GSSA keeps the incoming cases for `n`: the branch from `entry` selects the `a` region under `c1` and the `b` region under `not c1`. The local branches controlled by `c2` and `c3` go elsewhere, so they are not incoming cases for `n` and collapse to the `undef` fallback.

An `undef` value is an arbitrary value of the right type. In the VC, it is a fresh unconstrained symbol. It is used as the fallback arm that makes the gamma syntactically total, not as a demand-driven deletion of a verification-irrelevant case.

For a block b with direct controllers $c_1, \ldots, c_k$, the thin reachability gate is:

$$G_b = \vee_{i=1}^{k} \left(G_{c_i} \wedge \mathrm{orient}(c_i, b) \right)$$

For a value merge in b, the same direct-controller cases guard the incoming values, with an `undef` fallback:

$$x := \gamma\Big(K_{c_1,b}, v_1, \gamma\big(\ldots, \gamma\big(K_{c_k,b}, v_k, \mathrm{undef}\big)\big)\Big)$$

The thin construction therefore switches only on the direct-controller cases, but each case is guarded by both the controller's reachability and the controller's oriented branch condition. Bypass cases are outside the direct control-dependence cases for b, so value merges collapse them to `undef`.

Computing TGSSA

The thin conversion is a control-dependence pass:

1. Compute postdominators and the direct control-dependence relation.
2. For each block b, compute $\mathrm{ctrl}(b)$, the direct controller blocks sorted closest-to-b first.
3. For each controller $c \in \mathrm{ctrl}(b)$, compute $\mathrm{orient}(c, b)$ from the controller's branch condition.
4. Build G_b from the direct controller cases only, with `false` as the fallback case.
5. Materialize the required G_b variables in topological order.
6. Rewrite phi nodes and dynamic merge cases to use the materialized G_b variables.

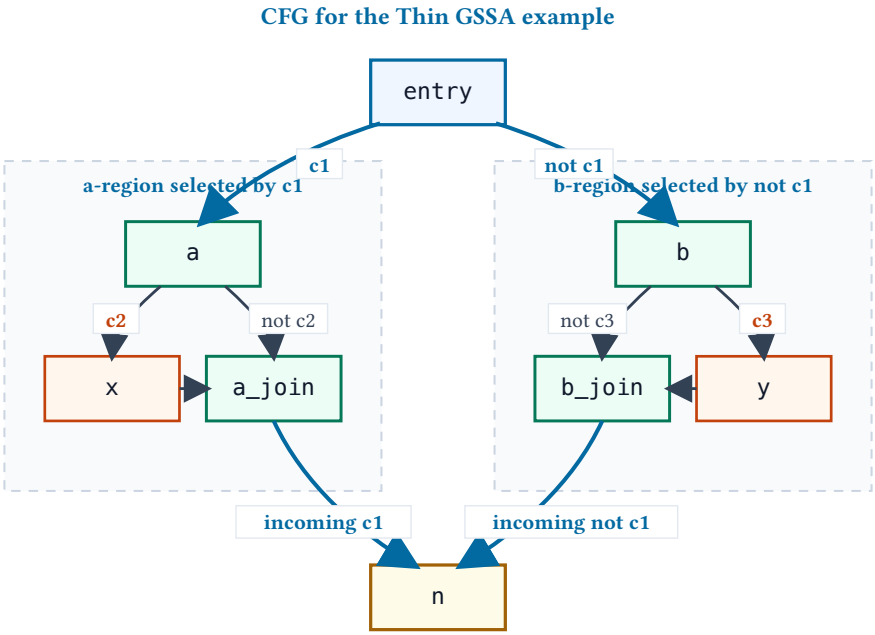


Figure 3: CFG for the Thin GSSA example. The incoming regions for the phi at `n` are selected by `c1` and `not c1`. The local branches `c2` and `c3` only choose paths inside those regions before they reconverge at `a_join` and `b_join`.

Example 2: Thin GSSA

The running diamond does not show the benefit: it has exactly two cases and no side branches. The benefit appears when one condition controls the incoming regions for a merge, while other branches remain local to those regions.

```
entry:
  c1 := havoc
  if c1 goto a else b

a:
  c2 := havoc
  v_a0 := 10
  if c2 goto x else a_join

x:
  v_x := 2
  goto a_join

a_join:
  v_a := phi [a: v_a0, x: v_x]
  goto n

b:
  c3 := havoc
  v_b0 := 20
```

```
  if c3 goto y else b_join

y:
  v_y := 3
  goto b_join

b_join:
  v_b := phi [b: v_b0, y: v_y]
  goto n

n:
  v := phi [a_join: v_a, b_join: v_b]
  assert v > 0
```

The phi at `n` merges the value produced by the `a` region with the value produced by the `b` region. The outer branch selects those regions: `c1` selects the `a` region, and `not c1` selects the `b` region. The local branches `c2` and `c3` only decide how each region computes its own value.

In regular GSSA, each phi is converted independently by using the full path condition for each incoming predecessor:

```
a_join:
  v_a := gamma(c1 and c2, v_x, gamma(c1 and not c2, v_a0, undef))

b_join:
  v_b := gamma(not c1 and c3, v_y,
         gamma(not c1 and not c3, v_b0, undef))

n:
  v := gamma(c1, v_a, gamma(not c1, v_b, undef))
```

In the last block, the left and right gate variables have already been inlined as `c1` and `not c1`. Since the source program is SESE, every real execution that reaches `n` arrives from exactly one predecessor. Thus the final `undef` fallback in the gamma for `n` is never selected on a real execution and can be dropped:

```
a_join:
  v_a := gamma(c2, v_x, gamma(not c2, v_a0, undef))

b_join:
  v_b := gamma(c3, v_y, gamma(not c3, v_b0, undef))

n:
  v := gamma(c1, v_a, v_b)
```

The same cleanup applies to the local phis, whose branch conditions are also complete two-way choices. The final form is:

```
a_join:
  v_a := gamma(c2, v_x, v_a0)
```

```
b_join:
  v_b := gamma(c3, v_y, v_b0)

n:
  v := gamma(c1, v_a, v_b)
```

Equivalently, after desugaring gamma:

```
a_join:
  v_a := ite(c2, v_x, v_a0)

b_join:
  v_b := ite(c3, v_y, v_b0)

n:
  v := ite(c1, v_a, v_b)
```

4.3.6 Cone of Influence

Once the program is in data-flow form, cone-of-influence reduction is simple. There are no CFG constraints to keep in sync with the program: every semantic dependency is an ordinary expression dependency. The reducer can start from the exit assumption and assertion and walk definitions backward.

The important point is that gates are ordinary Boolean definitions too. A gate definition is kept only when some retained gamma uses it. Branch conditions are then kept only when they are needed to define a retained gate. Branches that do not gate any retained value may remain unconstrained or be dropped from the data-flow formula.

COI Algorithm

Let defs map each SSA variable to its unique defining expression after phi nodes have been converted to gamma/ITE expressions. Let $\text{uses}(e)$ be the variables used by expression e.

1. Initialize the worklist with every variable used by the exit `assume` and `assert`.
2. While the worklist is non-empty, remove a variable x.
3. If x has no definition, keep its declaration and continue. This covers havoc inputs and `undef` symbols.
4. Keep the definition $x := e$.
5. Add every variable in $\text{uses}(e)$ to the worklist.
6. If e is a gamma/ITE whose guard is a gate G_b, mark G_b as needed.
7. Whenever a gate G_b is marked, keep its definition and recursively mark the gates and branch-condition variables used by that definition.
8. Drop every unmarked definition.

The output is the same data-flow program restricted to the retained definitions. Since every retained expression still names only retained dependencies or free inputs, the reduced VC has the same value for the exit assumption and assertion as the unreduced data-flow VC.

Example 3: COI Reduction
Consider the final Thin GSSA form above, with an extra value that is computed but never reaches the assertion:

```
a_join:
  v_a := ite(c2, v_x, v_a0)

b_join:
  v_b := ite(c3, v_y, v_b0)

n:
  v := ite(c1, v_a, v_b)
  junk := expensive(v_x, v_y)
  ok := v > 0

exit:
  assume true
  assert ok
```

The COI walk starts from `ok`. It keeps:

```
n:
  ok := v > 0
  v := ite(c1, v_a, v_b)

a_join:
  v_a := ite(c2, v_x, v_a0)

b_join:
  v_b := ite(c3, v_y, v_b0)
```

and drops:

```
n:
  junk := expensive(v_x, v_y)
```

because `junk` is not used by `ok`, the exit assumption, or any retained gate. No graph reachability rule is needed for this pruning; ordinary backwards data-dependence is enough after the GSSA conversion.

5 References and Borrowing

This section extends Tiny TAC with references. The goal is to model source languages that distinguish direct map access from access through borrowed locations, while keeping the VCGen interface explicit.

The extension is intentionally small:

- a reference names a location inside a bytemap,
- a constant reference may be read,
- a mutable reference may be read or written,
- references can be reborrowed from existing references,
- well-formed programs make the lifetime and aliasing obligations explicit enough for VC generation.

5.1 Reference Types

Tiny TAC gains reference types:

$$\tau ::= \text{bool} \mid \text{int} \mid \text{bytemap} \mid \& \ \tau \mid \&\text{mut} \ \tau$$

A value of type $\& \ \tau$ is a constant reference to a location containing a τ value. A value of type $\&\text{mut} \ \tau$ is a mutable reference to such a location.

For the base language in this document, the most important instances are references to integer cells inside a bytemap:

$$r : \&\text{int} \quad q : \&\text{mut int}$$

Symbolically, a reference carries three fields:

- the address to which the reference points,
- the value currently observed through the reference,
- the promised value at that address when the reference is released.

For the lowering below, we represent each reference as a triple:

$$r = \{\text{addr} : i, \text{value} : v, \text{promise} : p\}$$

The `addr` field is the address to which the reference points. The `value` field is the value stored at that address from the reference's point of view. The `promise` field is the value that will be stored at that address when the reference is released. Constant references do not use their promise field, but we keep the same triple shape for simplicity.

Tiny TAC does not otherwise have tuples or field projection. In this section, record literals such as `{ addr: i, value: v, promise: p }` and projections such as `r.value` are syntactic sugar used only to explain the lowering.

5.2 Borrowing Commands

The command grammar is extended with explicit borrow, reborrow, `get_ref`, `put_ref`, and release commands:

```
cmd ::= ...
      | r := borrow M[i]
      | r, M2 := borrow_mut M[i]
      | q := borrow_ref r
      | q, r2 := borrow_ref_mut r
      | x := get_ref r
      | r2 := put_ref r, v
      | release r
```

The first two commands borrow a concrete memory location. The next two commands reborrow the location described by an existing reference. `get_ref` observes the value through a reference. `put_ref` updates the value through a mutable reference and returns a fresh reference value. `release` ends the reference lifetime for the purposes of well-formedness checking.

A constant borrow returns only the new reference because it cannot modify the underlying bytemap. A mutable borrow returns the new reference and a continuation bytemap that records any potential modification through the borrowed reference. Mutable reborrowing similarly returns the new reference

`q` and a continuation reference `r2` that represents the original reference after the borrow of `q` ends. This keeps the program in SSA form even when a borrow may mutate through the borrowed reference.

For example:

```
entry:
  M := havoc
  i := havoc
  p := borrow M[i]
  x := get_ref p
  release p
  q, M2 := borrow_mut M[i]
  q2 := put_ref q, (x + 1)
  release q2
  ok := M2[i] == x + 1
  assert ok
  halt
```

This program first creates a constant reference for reading, releases it, and then creates a mutable reference to update the same location.

5.3 Reborrowing

Reborrowing creates a new reference to the same location as an existing reference. The surface syntax is:

```
p := borrow M[i]
p2 := borrow_ref p
x := get_ref p2
release p2
release p
```

A mutable reference can also be reborrowed:

```
r, M1 := borrow_mut M[i]
q, q_after := borrow_ref_mut r
q2 := put_ref q, 7
release q2
x := get_ref q_after
ok := x == 7
assert ok
release q_after
```

The well-formedness condition is stronger for mutable reborrows: while `q` is live, the parent reference `r` is suspended. After the borrowed reference is released, `q_after` represents the resumed parent reference and can observe the update performed through `q`.

5.4 Borrow Well-Formedness

Borrowing introduces obligations that are best checked before VC generation. We expect conditions such as: `get_ref` requires a live reference, `put_ref` requires a live mutable reference, mutable references

are exclusive in the appropriate sense, and reborrows do not outlive the references from which they were derived. This document does not attempt to define the exact borrow discipline.

It is also legal in Tiny TAC to mix direct memory operations with reference operations:

```
r, M1 := borrow_mut M[i]
r2 := put_ref r, 7
M2 := M1[j := 9]
release r2
```

This is analogous to a Rust program that mixes safe references with unsafe raw accesses. The correct well-formedness conditions for such programs depend on the chosen reference semantics.

One possible family of rules follows *Stacked Borrows*: (see, stacked-borrows.md). The Rust unsafe-code-guidelines notes describe it as a non-normative model and link to the Stacked Borrows paper and related material. Another possible model is *Tree Borrows*: (see tree-borrows.md).

The rest of this document treats borrow well-formedness as an external side condition or as an optional set of additional VC obligations, without choosing between these models.

5.5 Compiling References Away

We can explain VC generation for references by first compiling Tiny TAC with references into ordinary Tiny TAC plus the tuple notation introduced above. After this source-to-source lowering, the existing VCGen only needs to handle ordinary assignments, map reads, map updates, assumptions, and assertions.

For a constant borrow:

```
r := borrow M[i]
```

the lowering is:

```
r := { addr: i, value: M[i], promise: havoc }
```

The `promise` field is unused for constant references, but keeping it makes all references have the same shape.

For a mutable borrow:

```
r, M2 := borrow_mut M[i]
```

the lowering is:

```
r := { addr: i, value: M[i], promise: havoc }
M2 := M[i := r.promise]
```

The map `M2` is the continuation memory: when `r` is eventually released, the borrow promises that address `i` contains `r.promise`.

For a `get_ref`:

```
x := get_ref r
```

the lowering is:

```
x := r.value
```

For a `put_ref` through a mutable reference:

```
r2 := put_ref r, v
```

the lowering is:

```
r2 := { addr: r.addr, value: v, promise: r.promise }
```

The `put_ref` returns a fresh reference symbol so the lowered program remains in SSA form.

For a release:

```
release r
```

the lowering is:

```
assume r.value == r.promise
```

This assumption connects the value observed through the reference at release time with the promise that was already written into the continuation memory.

5.5.1 Example: Direct Mutable Borrow
Consider this Tiny TAC program with references:

```
entry:
  M := havoc
  i := havoc
  r, M2 := borrow_mut M[i]
  r2 := put_ref r, 7
  release r2
  x := M2[i]
  ok := x == 7
  assert ok
  halt
```

The reference commands compile to:

```
entry:
  M := havoc
  i := havoc
  r := { addr: i, value: M[i], promise: havoc }
  M2 := M[i := r.promise]
  r2 := { addr: r.addr, value: 7, promise: r.promise }
```

```
  assume r2.value == r2.promise
  x := M2[i]
  ok := x == 7
  assert ok
  halt
```

After substituting the tuple fields, the relevant facts are:

$$\begin{aligned} \mathrm{M2} &= M[i := \mathrm{promise}(r)] \\ \mathrm{value}(\mathrm{r2}) &= 7 \\ \mathrm{promise}(\mathrm{r2}) &= \mathrm{promise}(r) \\ \mathrm{value}(\mathrm{r2}) &= \mathrm{promise}(\mathrm{r2}) \end{aligned}$$

Together these imply $\mathrm{promise}(r) = 7$, so the continuation memory `M2` stores `7` at address `i`. The final assertion is therefore the ordinary Tiny TAC fact that `M2[i] == 7`.

5.6 Compiling Borrowed References

Reborrowing is also compiled away using the same reference triple. A constant reborrow:

```
q := borrow_ref r
```

compiles to:

```
q := { addr: r.addr, value: r.value, promise: havoc }
```

The child reference `q` observes the same address and current value as `r`. Since `q` is constant, its promise is unused.

A mutable reborrow:

```
q, r2 := borrow_ref_mut r
```

compiles to:

```
q := { addr: r.addr, value: r.value, promise: havoc }
r2 := { addr: r.addr, value: q.promise, promise: r.promise }
```

When `q` is released, it updates the value of `r2`. When `r2` is released, it updates the value that `r` promised to release. At the same time, `r` is consumed and converted to `r2`, so `r` itself is never released.

5.6.1 Example: Mutable Reborrow

Consider:

```
entry:
  M := havoc
  i := havoc
  r, M2 := borrow_mut M[i]
```

```
  q, r2 := borrow_ref_mut r
  q2 := put_ref q, 7
  release q2
  r3 := put_ref r2, 8
  release r3
  x := M2[i]
  ok := x == 8
  assert ok
  halt
```

The reference commands compile to:

```
entry:
  M := havoc
  i := havoc
  r := { addr: i, value: M[i], promise: havoc }
  M2 := M[i := r.promise]
  q := { addr: r.addr, value: r.value, promise: havoc }
  r2 := { addr: r.addr, value: q.promise, promise: r.promise }
  q2 := { addr: q.addr, value: 7, promise: q.promise }
  assume q2.value == q2.promise
  r3 := { addr: r2.addr, value: 8, promise: r2.promise }
  assume r3.value == r3.promise
  x := M2[i]
  ok := x == 8
  assert ok
  halt
```

The first release gives `q.promise == 7`, so `r2.value == 7`. The update through `r2` then changes the value to `8` while preserving `r.promise` as the final promise. Releasing `r3` gives `r.promise == 8`, which is the value already written into the continuation memory `M2`.

The relevant equalities are:

$$
\begin{aligned}
\mathrm{M2} &= M[i := \mathrm{promise}(r)] \\
\mathrm{promise}(\mathrm{r2}) &= \mathrm{promise}(r) \\
\mathrm{value}(\mathrm{q2}) &= 7 \\
\mathrm{promise}(\mathrm{q2}) &= \mathrm{promise}(q) \\
\mathrm{value}(\mathrm{q2}) &= \mathrm{promise}(\mathrm{q2}) \\
\mathrm{value}(\mathrm{r3}) &= 8 \\
\mathrm{promise}(\mathrm{r3}) &= \mathrm{promise}(\mathrm{r2}) \\
\mathrm{value}(\mathrm{r3}) &= \mathrm{promise}(\mathrm{r3})
\end{aligned}
$$

These imply $\mathrm{promise}(r) = 8$, so `M2[i] == 8`. The value written by `q` is visible through `r2`, but the final update through `r2` determines the value that the original mutable borrow commits to memory.

5.7 VCGen Extension

With this lowering, references do not require a separate VCGen core rule in the direct-borrow and reborrow cases above. VCGen can first compile reference commands away, then run the ordinary Tiny TAC encoding on the resulting program.

5.8 Extensions and Discussion

The `promise` field is an instance of a *prophecy variable*: a nondeterministic value chosen now and constrained later when the promised event occurs. Prophecy variables were introduced by Abadi and Lamport in *The Existence of Refinement Mappings* in the context of proving refinement between concurrent and reactive system specifications. Later presentations, such as Lamport and Merz's *Auxiliary Variables in TLA+*, describe prophecy variables together with history and stuttering variables as auxiliary variables that can be added to a specification without changing its observable behaviors.

The same idea appears in verification of concurrent data structures and model checking when a proof needs to refer to a value that will only be determined by a future step. In the borrow encoding above, the future step is `release`: the borrow begins by choosing a promised final value, and the release later fulfills that promise.

The use of prophecy-style values to model Rust borrows is due to RustHorn: *RustHorn: CHC-based Verification for Rust Programs* by Matsushita, Tsukada, and Kobayashi. RustHorn represents a mutable reference using the current value and the value at the end of the borrow, avoiding an explicit memory model for safe Rust-style ownership.

Priya and Gurfinkel adapt this idea to a bounded-model-checking setting in *Ownership in low-level intermediate representation* (FMCAD 2024). Their setting is lower-level than RustHorn: ownership information is used opportunistically in an LLVM-like IR, while unsafe or raw-style accesses may still require synchronization with an address-map memory model.

The Tiny TAC presentation here follows the same broad pattern, but remains only a small explanatory model. The next questions are how to combine the prophecy translation with the chosen borrow well-formedness discipline, how to handle mixed direct-memory and reference accesses, and how much of the resulting translation should be treated as preprocessing versus part of VCGen proper.

Several extensions are left open. Each is small enough to state independently, but substantial enough to be a useful project.

- **Finite-region borrows.** The examples restrict memory borrows to a single memory cell. A natural extension is to borrow an arbitrary finite region, such as a collection of bytes. The reference representation would then need to describe a base address, a size or set of offsets, and a value/promise for each byte or word in the region. The main question is how to keep the resulting encoding compact while still allowing accesses inside the borrowed region to use the reference representation instead of the full memory map.
- **Reference shadow memory.** Tiny TAC does not allow references to be stored in memory, because a reference triple does not fit into a single `int`. One way to model stored references is to add ghostmaps for shadow memory. For every ordinary memory byte, the encoding could maintain corresponding shadow values that store reference metadata such as address, value, promise, permission, or validity. The design question is which metadata must be shadowed, and how shadow updates stay synchronized with ordinary memory updates.

- **Reference semantics and well-formedness.** This section has not given a precise semantics for references, especially when reference operations are mixed with raw memory accesses. It also has not specified how to check borrow well-formedness. A project in this direction could choose a model, such as a Stacked Borrows or Tree Borrows variant, state the well-formedness rules, and decide whether violations are rejected before VCGen or encoded as additional verification obligations.